TRABALHO PRÁTICO

Estacionamento em uma via

Árvores balanceadas (AVL ou rubro-negra) e tabela hash em C

Curso/turma: *preencher* **Entrega até:** *preencher* **Valor:** *preencher* **Formação:** *individual/dupla*

1. O problema

Faça um programa em C para gerenciar o estacionamento de veículos em uma via com M metros, considerando as seguintes características do problema:

- Inicialmente não há veículo estacionado na via, ou seja, há uma única vaga com tamanho M metros;
- Os veículos só podem ser estacionados linearmente, nunca lado a lado ou sobre outro;
- Cada veículo possui uma identificação única e ocupa um espaço de m metros, já considerando o espaço necessário para a manobra de entrada e saída da vaga. O espaço m é informado com duas casas decimais (precisão de centímetro).

2. Requisitos do programa

- Inicialmente o usuário informa o comprimento M da via. Em seguida, interage com o programa solicitando cada operação desejada: estacionamento de veículo, saída de veículo ou término do programa;
- O usuário sempre deve ser informado se uma solicitação de estacionamento foi atendida com sucesso ou não. Quando um estacionamento não puder ser realizado e o tamanho do veículo não exceder M, esse veículo aguardará a oportunidade de estacionar em uma fila. Sempre que existir vaga compatível com o veículo que aguarda há mais tempo, o programa perguntará ao usuário se esse veículo deve ser estacionado ou se continuará aguardando na fila;
- O estacionamento de um veículo deve ocorrer sempre no início da menor vaga que o comporte;
- Todos os veículos, estacionados ou aguardando na fila, devem ser armazenados em uma **tabela hash** indexada pela identificação do veículo, com tratamento de colisões por encadeamento separado e função de dispersão pelo método da divisão, h(k) = k mod m, com m primo. A identificação é um número natural, informado pelo usuário junto com o tamanho do veículo na solicitação de estacionamento. Não pode haver dois veículos com a mesma identificação, nem na via nem na fila;
- As vagas disponíveis devem ser armazenadas em uma **árvore balanceada ordenada pelo tamanho** da vaga e, em caso de empate, pela posição inicial. A busca pela menor vaga que comporte o veículo deve ser feita descendo a árvore, sem percorrê-la por completo;
- Todos os trechos da via, livres e ocupados, devem ser armazenados em uma **árvore balanceada ordenada pela posição inicial** do trecho. Quando um veículo sai, a vaga liberada deve ser mesclada com as vagas adjacentes, se houver, formando uma única vaga com o tamanho total. Essas vagas adjacentes são o antecessor e o sucessor do trecho nessa árvore. As duas árvores devem ser atualizadas corretamente a cada estacionamento e a cada saída;
- O usuário deve informar apenas a identificação do veículo para solicitar a saída. Se a identificação não corresponder a um veículo estacionado, o usuário deve ser avisado;

- Todas as estruturas de dados devem ser devidamente desalocadas ao final do uso.

3. Escolha da árvore balanceada

Cada aluno escolhe se vai implementar **árvore AVL** ou **árvore rubro-negra**. A escolha vale para as duas árvores do trabalho e deve ser informada no comentário de cabeçalho do arquivo principal.

O ideal é escrever uma única implementação da árvore e usá-la duas vezes, trocando só a função que compara as chaves (por tamanho e posição em uma, por posição na outra). Duplicar o código também é aceito, desde que o balanceamento esteja correto nas duas. A remoção com rebalanceamento é obrigatória qualquer que seja a escolha, porque as duas árvores perdem nós durante a execução: a de tamanhos a cada estacionamento e a cada mescla, a de posições a cada mescla.

4. O que manter em tela

- Um mapa linear do estacionamento, indicando vagas disponíveis (mostrar o tamanho) e ocupadas (mostrar a identificação), obtido pelo percurso em ordem da árvore de posições;
- A porcentagem de ocupação da via;
- A tabela hash, com a lista de colisões de cada posição, e o fator de carga atual;
- A árvore de vagas disponíveis, mostrando para cada nó a altura e o fator de balanceamento (AVL) ou a cor (rubro-negra);
- A fila de veículos aguardando estacionamento.

O código deve estar devidamente organizado, indentado, comentado e sem erros ou avisos usando as opções `-Wall -Wextra -pedantic` do gcc.

5. Como as estruturas trabalham juntas

As figuras desta seção acompanham o exemplo de execução da seção 6, com uma via de 20,00 m. A maioria mostra o estado da via logo depois da operação 8, quando já há três vagas livres e duas delas têm o mesmo tamanho.

O mapa da via

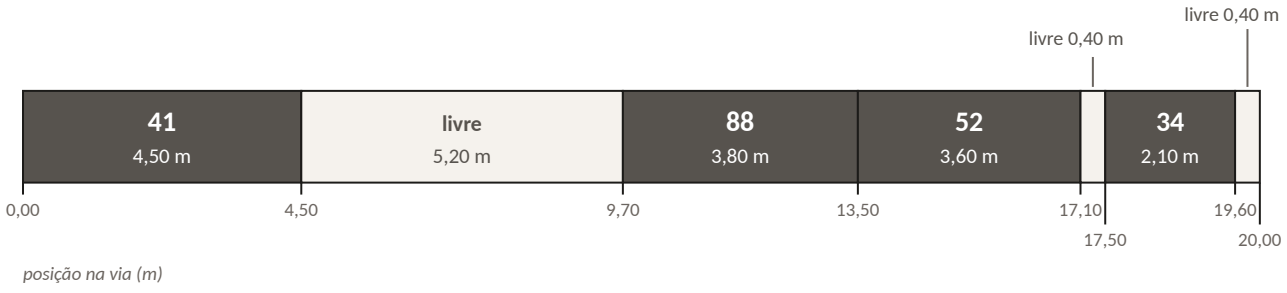


Figura 1. Via de 20,00 m depois da operação 8. Os trechos escuros estão ocupados e mostram a identificação do veículo; os claros são vagas livres. Ocupação: 14,00 m de 20,00 m, ou seja, 70,00%.

Na tela, o mapa pode ser impresso de forma textual. Por exemplo:

```
Via: 20,00 m    Ocupação: 70,00%
[41 4,50][livre 5,20][88 3,80][52 3,60][livre 0,40][34 2,10][livre 0,40]
```

A árvore de posições

Cada nó é um trecho da via, com a posição inicial como chave. O percurso em ordem devolve os trechos da esquerda para a direita, que é exatamente o mapa da Figura 1. As duas figuras abaixo mostram a mesma árvore nas duas versões possíveis do trabalho. A forma real depende da ordem das operações e da implementação; o que importa é que a árvore seja válida.

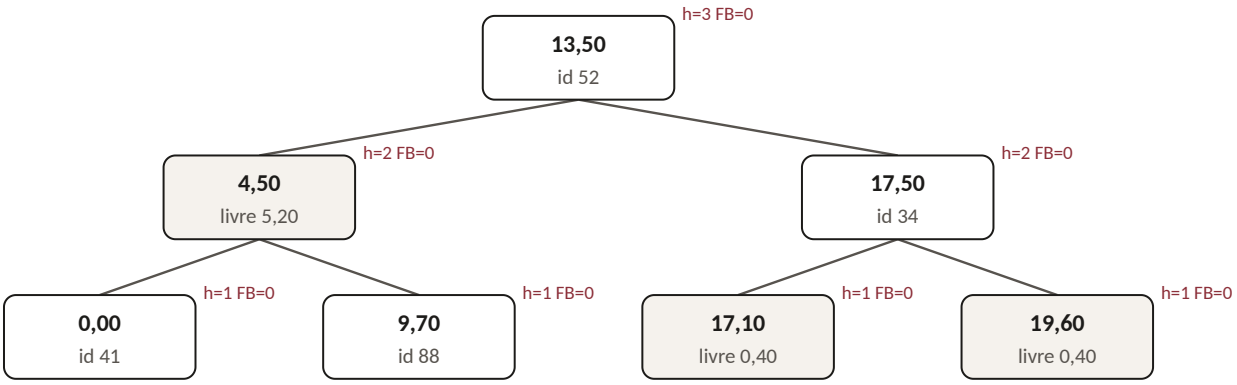


Figura 2a. Versão AVL. Ao lado de cada nó, a altura (folha com altura 1) e o fator de balanceamento. Nós com fundo claro são vagas livres.

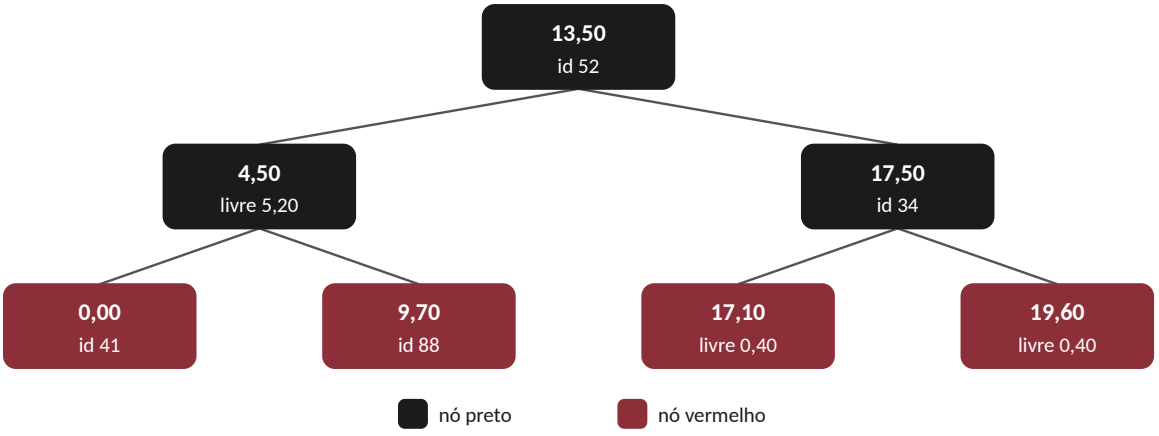


Figura 2b. Versão rubro-negra, com uma coloração válida para a mesma forma. As folhas nulas (NIL), todas pretas, foram omitidas.

A árvore de vagas e a busca pela menor vaga

Na árvore de vagas a chave é o par (tamanho, posição inicial). Para estacionar um veículo de tamanho t, o programa procura a menor chave com tamanho maior ou igual a t. A descida funciona assim: se o nó atual comporta o veículo, ele vira o candidato e a busca continua à esquerda, procurando algo menor que ainda sirva; se não comporta, a busca continua à direita. Quando a descida chega a um ponteiro nulo, o último candidato é a resposta. Se não houve candidato, nenhuma vaga serve e o veículo vai para a fila.

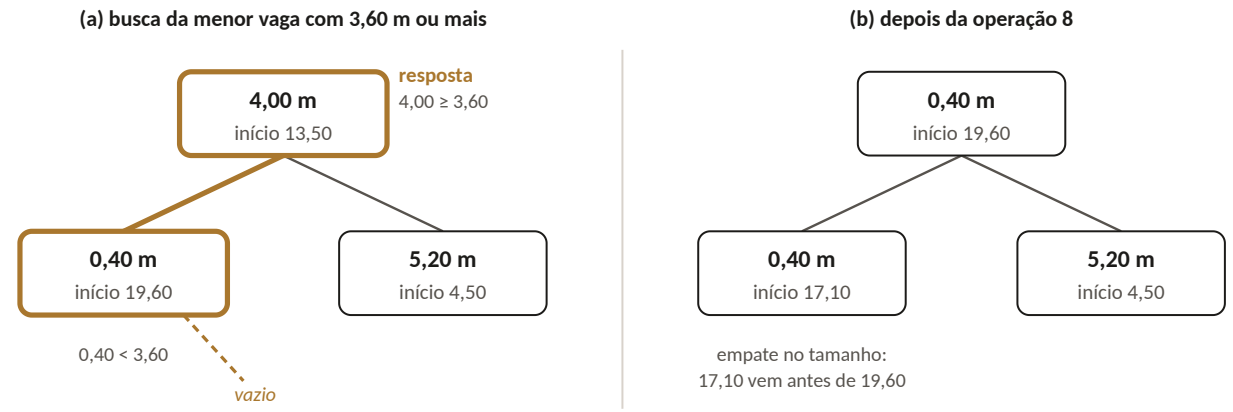


Figura 3. Em (a), a árvore antes da operação 8 e o caminho percorrido para estacionar o veículo 52, de 3,60 m. Em (b), a árvore depois que a vaga (4,00; 13,50) foi retirada e a sobra (0,40; 17,10) foi inserida. As duas vagas de 0,40 m só podem ser distinguidas pela posição, por isso ela faz parte da chave.

A mescla na saída de um veículo

Na operação 10, o veículo 34 sai. O programa localiza o veículo na tabela hash, obtém sua posição inicial (17,50) e encontra o nó correspondente na árvore de posições. O antecessor desse nó é a vaga livre que começa em 17,10 e o sucessor é a vaga livre que começa em 19,60. Como as duas estão livres, os três trechos viram um só.

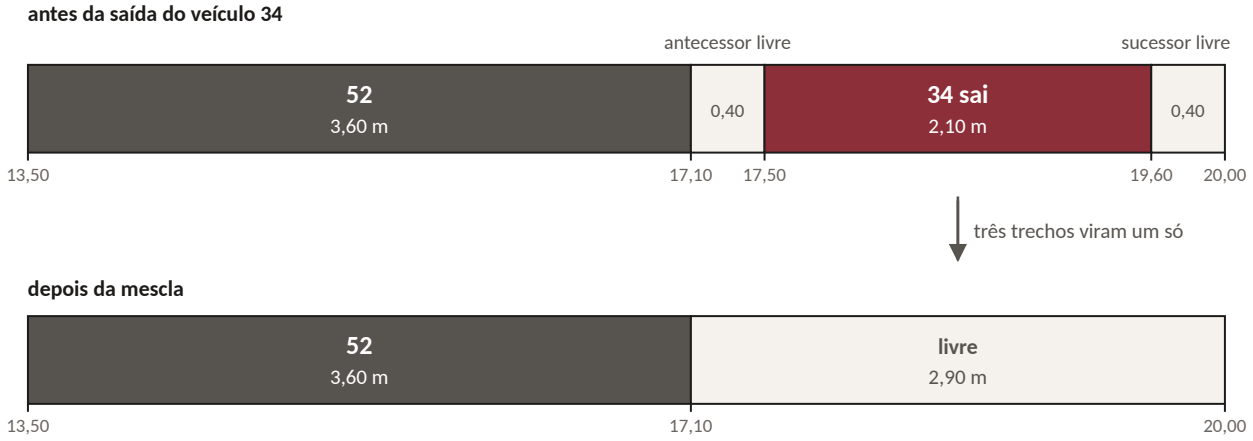


Figura 4. Trecho final da via (de 13,50 a 20,00 m, em escala) antes e depois da saída do veículo 34.

Do lado das estruturas, essa saída provoca: a remoção do 34 da tabela hash; a remoção dos nós 17,50 e 19,60 da árvore de posições e a atualização do nó 17,10, que passa a medir 2,90 m; a remoção das chaves (0,40; 17,10) e (0,40; 19,60) da árvore de vagas e a inserção de (2,90; 17,10). Por fim, o programa consulta a fila: o veículo 30 precisa de 6,00 m e continua aguardando.

A tabela hash

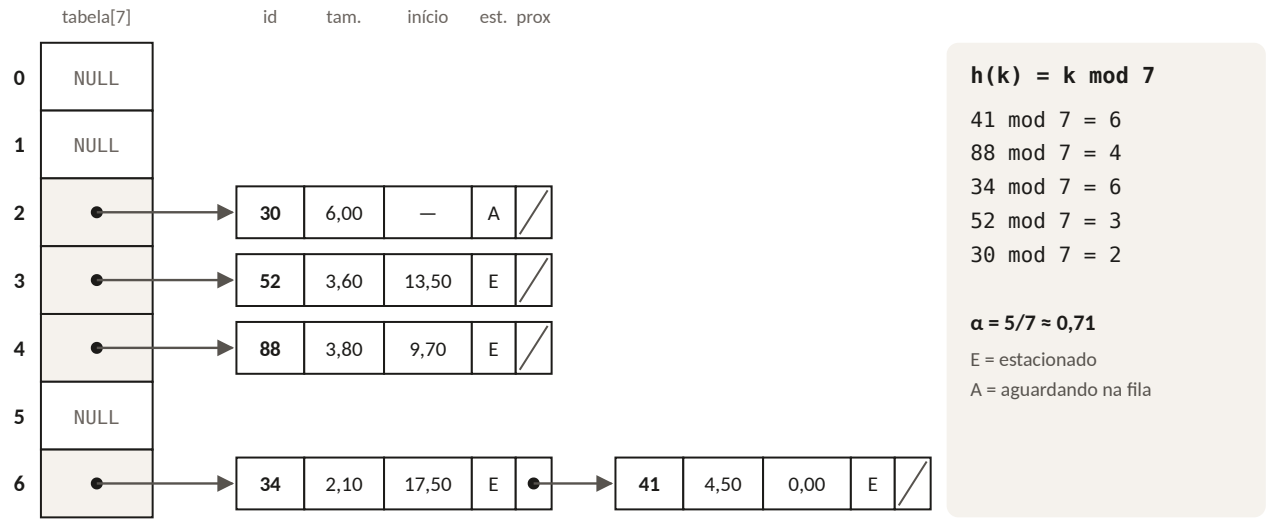


Figura 5. Tabela hash com m = 7 depois da operação 9, quando o veículo 30 entrou na fila. Os veículos 41 e 34 colidem na posição 6; como cada novo registro é inserido no início da lista, o 34, que chegou depois, aparece primeiro. Cinco registros em sete posições dão fator de carga de aproximadamente 0,71. O valor m = 7 foi escolhido só para a figura; no programa, o tamanho da tabela fica a critério do aluno, desde que seja primo.

6. Exemplo de execução

Via com M = 20,00 m. Nas vagas livres, cada item aparece como tamanho (posição inicial), já na ordem da árvore de vagas. Medidas em metros.

Nº	Operação	Resultado	Vagas livres	Ocupação
1	estacionar 41, 4,50 m	estacionado em 0,00	15,50 (4,50)	22,50%
2	estacionar 17, 5,20 m	estacionado em 4,50	10,30 (9,70)	48,50%
3	estacionar 88, 3,80 m	estacionado em 9,70	6,50 (13,50)	67,50%
4	estacionar 25, 4,00 m	estacionado em 13,50	2,50 (17,50)	87,50%
5	estacionar 34, 2,10 m	estacionado em 17,50	0,40 (19,60)	98,00%
6	saída 17	libera 4,50 a 9,70; os vizinhos estão ocupados, não há mescla	0,40 (19,60) · 5,20 (4,50)	72,00%
7	saída 25	libera 13,50 a 17,50; também sem mescla	0,40 (19,60) · 4,00 (13,50) · 5,20 (4,50)	52,00%
8	estacionar 52, 3,60 m	a menor vaga que cabe é a de 4,00 m; estacionado em 13,50	0,40 (17,10) · 0,40 (19,60) · 5,20 (4,50)	70,00%
9	estacionar 30, 6,00 m	nenhuma vaga comporta o veículo; ele entra na fila	sem alteração	70,00%
10	saída 34	libera 17,50 a 19,60 e mescla com as duas vagas vizinhas; o 30 ainda não cabe	2,90 (17,10) · 5,20 (4,50)	59,50%
11	saída 88	libera 9,70 a 13,50 e mescla com a vaga da esquerda; agora o 30 cabe e o programa pergunta ao usuário	2,90 (17,10) · 9,00 (4,50)	40,50%
12	usuário confirma	30 sai da fila e é estacionado em 4,50	2,90 (17,10) · 3,00 (10,50)	70,50%

7. Observações de implementação

- **Trabalhe com centímetros inteiros.** Leia o tamanho como `double` e converta logo na entrada, por exemplo com `(int)(m * 100.0 + 0.5)`. Comparar `float` ou `double` com == para decidir se duas vagas são vizinhas dá errado por arredondamento.
- **Não guarde ponteiro para nó de árvore** dentro de outra estrutura. Muitas implementações de remoção copiam a chave do sucessor para o nó removido, e qualquer ponteiro guardado passa a apontar para o trecho errado. Guarde no registro do veículo a posição inicial e busque na árvore de posições quando precisar.
- Antecessor e sucessor podem ser encontrados com ponteiro para o pai em cada nó ou com uma busca a partir da raiz. As duas formas são aceitas.
- Quando um veículo sai da fila e estaciona, o registro dele na tabela hash muda de estado; ele não deve ser removido e inserido de novo.
- Teste com uma tabela pequena para provocar colisões e com sequências que forcem rotações nas duas árvores, inclusive na remoção.
- O programa deve compilar com `gcc -Wall -Wextra -pedantic` sem nenhum aviso.

8. Entrega

O que entregar: *preencher (ex.: arquivos .c e .h compactados)*

Onde entregar: *preencher*

Critérios de avaliação: *preencher*

Item extra (opcional): redimensionar a tabela hash para o próximo primo maior que o dobro do tamanho atual sempre que o fator de carga passar de 0,75, reinserindo todos os registros. *pontuação extra: preencher*